ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 2ND YEAR

OPERATING SYSTEMS

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 2nd year Operating Systems paper 1 · practice a.

Written by Academic Central for practice, following second-year Computer Science syllabuses at South African universities (operating systems). It is not a university memorandum.

How to mark: each ✓ is one mark. Give marks for correct working even if a later step is wrong (consistent accuracy). Accept other correct pseudocode and equivalent wording.

QUESTION 1

1.1 A ✓
1.2 C ✓
1.3 C ✓
1.4 A ✓
1.5 A ✓
1.6 B ✓
1.7 A ✓
1.8 A ✓
1.9 A ✓
1.10 B ✓

(10)

[10]

QUESTION 2

2.1 A program is a passive set of instructions stored on disk ✓; a process is a program in execution, with its own memory, registers and state ✓ (2)

2.2 States: New, Ready, Running, Blocked (Waiting), Terminated ✓
Transitions: New → Ready (admitted) ✓; Ready → Running (scheduler dispatch) ✓; Running → Ready (interrupt or time slice over) and Running → Blocked (waits for I/O or an event) ✓; Blocked → Ready (I/O or event finished) and Running → Terminated (exit) ✓ (5)

2.3 Any THREE: process ID ✓; process state ✓; program counter ✓; CPU registers; scheduling information (priority); memory-management information (page table); list of open files; accounting information (3)

2.4 Any TWO: threads share memory, so communication is easy and fast ✓; creating and switching threads is cheaper than for processes ✓; a program can stay responsive while one thread waits; threads can run on several cores at the same time (2)

2.5 A request from a program to the operating system kernel for a service it may not do itself, such as I/O ✓✓. Example, any ONE: read(), write(), open(), fork(), exec(), exit() ✓ (3)

[15]

QUESTION 3

3.1 Gantt chart: [P1 0–8] [P2 8–12] [P3 12–21] [P4 21–26] ✓✓
Waiting: P1 = 0, P2 = 8 − 1 = 7, P3 = 12 − 2 = 10, P4 = 21 − 3 = 18 ✓
Average = 35 / 4 ✓ = 8.75 ms ✓ (5)

3.2 Gantt chart: [P1 0–8] [P2 8–12] [P4 12–17] [P3 17–26] ✓✓ (only P1 has arrived at 0; at 8 the shortest is P2, then P4)
Waiting: P1 = 0, P2 = 7, P4 = 12 − 3 = 9, P3 = 17 − 2 = 15 ✓
Average = 31 / 4 ✓ = 7.75 ms ✓ (5)

3.3 Gantt chart: [P1 0–3] [P2 3–6] [P3 6–9] [P4 9–12] [P1 12–15] [P2 15–16] [P3 16–19] [P4 19–21] [P1 21–23] [P3 23–26] ✓✓✓
Finish times: P1 23, P2 16, P3 26, P4 21
Waiting = finish − arrival − burst: P1 = 15, P2 = 11, P3 = 15, P4 = 13 ✓✓
Average = 54 / 4 = 13.5 ms ✓ (6)

3.4 SJF (7.75 ms) ✓. The OS does not know the length of the next CPU burst in advance; it can only estimate it ✓ (2)

3.5 Pre-emptive: the OS can take the CPU away from a running process (e.g. when its time slice ends or a higher-priority process arrives) ✓
Non-pre-emptive: a process keeps the CPU until it finishes or blocks ✓ (2)

[20]

QUESTION 4

4.1 A race condition happens when the result depends on the order in which threads access shared data ✓. counter++ is really: read counter, add 1, write back ✓. If both threads read 5 before either writes, both write 6, and one increment is lost (the answer should be 7) ✓ (3)

4.2 Mutual exclusion ✓ Progress ✓ Bounded waiting ✓ (3)

4.3 Model answer: (6)

```
Producer:                          Consumer:
  loop                               loop
    produce item                       wait(full)        ✓ waits for an item
    wait(empty)      ✓                 wait(mutex)       ✓
    wait(mutex)      ✓                 remove item from buffer
    add item to buffer                 signal(mutex)
    signal(mutex)                      signal(empty)     ✓ frees a slot
    signal(full)     ✓                 consume item
                                                         ✓ correct order: wait(empty/full) BEFORE wait(mutex)
```

4.4 Mutual exclusion ✓ Hold and wait ✓ No pre-emption ✓ Circular wait ✓ (4)

4.5 Need = Max − Allocation: P0 (2, 2), P1 (1, 1), P2 (3, 1) ✓ (4)
Work = (2, 1): P1 can finish (1, 1) ≤ (2, 1); Work = (2, 1) + (2, 1) = (4, 2) ✓
P0 can finish (2, 2) ≤ (4, 2); Work = (5, 2). P2 can finish (3, 1) ≤ (5, 2); Work = (6, 3) ✓
The state is SAFE; safe sequence P1, P0, P2 ✓

[20]

QUESTION 5

5.1 Internal fragmentation: wasted space INSIDE an allocated block, because the block is bigger than needed (e.g. the last page of a process) ✓ (2)
External fragmentation: enough free memory in total, but split into small gaps BETWEEN allocated blocks, so a large request cannot fit ✓

5.2 Offset: 1 024 = 2^{10}, so 10 bits ✓ Page number: 16 − 10 = 6 bits ✓ Pages: 2^6 = 64 ✓ (3)

5.3 Page = 2 100 ÷ 1 024 = 2, offset = 2 100 − 2 048 = 52 ✓ (3)
Page 2 is in frame 7 ✓
Physical address = 7 × 1 024 + 52 = 7 220 ✓

5.4.1 FIFO frames after each reference: [7] [7,0] [7,0,1] [0,1,2] [0,1,2] [1,2,3] [2,3,0] [3,0,4] [0,4,2] [4,2,3] ✓✓✓ (4)
Page faults = 9 (only the second reference to 0 is a hit) ✓

5.4.2 LRU: [7] [7,0] [7,0,1] [0,1,2] (7 out) hit on 0, [0,2,3] (1 out) hit on 0, [0,3,4] (2 out), [0,4,2] (3 out), [4,2,3] (0 out) ✓✓✓ (4)
Page faults = 8 ✓

5.5 A small, fast hardware cache of recent page-table entries ✓; it lets most address translations be done without reading the page table in memory, making memory access faster ✓ (2)

5.6 For some reference strings, FIFO gives MORE page faults when it is given MORE frames ✓✓ (2)

[20]

QUESTION 6

6.1 Contiguous: fast sequential and direct access ✓, but external fragmentation and files are hard to grow ✓ (6)
Linked: no external fragmentation and files grow easily ✓, but direct access is slow and a broken pointer loses the rest of the file ✓
Indexed: fast direct access and no external fragmentation ✓, but the index block uses extra space ✓

6.2.1 Order: 82, 170, 43, 140, 24, 16, 190 ✓ (3)
32 + 88 + 127 + 97 + 116 + 8 + 174 ✓ = 642 cylinders ✓

6.2.2 Order: 43, 24, 16, 82, 140, 170, 190 ✓ (3)
7 + 19 + 8 + 66 + 58 + 30 + 20 ✓ = 208 cylinders ✓

6.3 Any THREE: file name ✓; size ✓; owner and permissions ✓; type; dates created, modified and last accessed; location of the data blocks (pointers) (3)

[15]

TOTAL: 100

www.academiccentral.co.za

END OF MARKING GUIDELINE